

Monte Carlo Tree Search

RLSS 2026 Tutorial

RLSS Staff

June 4th, 2026



UNIVERSITÀ
DEGLI STUDI
DI MILANO

Learning goals

- Understand Monte Carlo Tree Search as a simulator-based planning method
- Explain the four MCTS phases: selection, expansion, rollout, backpropagation
- Implement UCB action selection and updates of node-level statistics
- Apply MCTS to Gymnasium's FrozenLake-v1 environment
- Visualise a planned policy on the lake grid

Why planning?

Core planning question

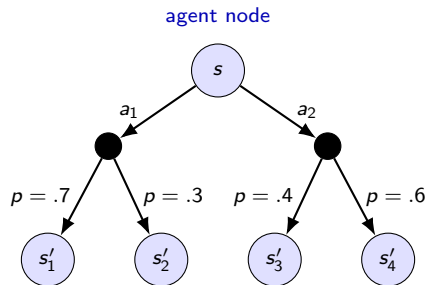
Which action should the agent take now to maximise future reward?

- If we have a model or simulator, we can *think before acting*
- Exhaustive search is often impossible: the tree grows exponentially with horizon
- MCTS spends simulations selectively on promising trajectories
- It is an *anytime* algorithm: more simulations usually improve decisions



Game trees for planning

- A *game tree* unfolds possible futures from the current state.
- In our MDP view, the two “players” alternate:
 - the learner/agent chooses action a
 - the environment samples the next state $s' \sim P(\cdot \mid s, a)$
- Planning estimates which root action has the best expected future return.
- MCTS grows only the parts of this tree that simulations visit.



Where does MCTS appear?

- Board and combinatorial games
- Robotics and control with simulators
- Planning in learned world models
- Program synthesis and sequence design

Main ingredient

We need a way to sample: if I take action a in state s , what might happen next?

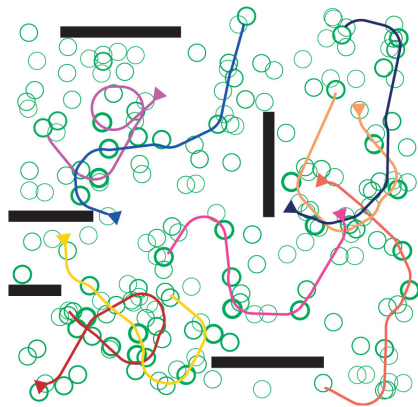


Image credit: Best et al. (2018), "Dec-MCTS: Decentralized planning for multi-robot active perception", IJRR.

FrozenLake as a small MDP

Problem: A Christmas gift ended up on a frozen lake and one of Santa's helpers is trying to retrieve it. Watch out for the holes!

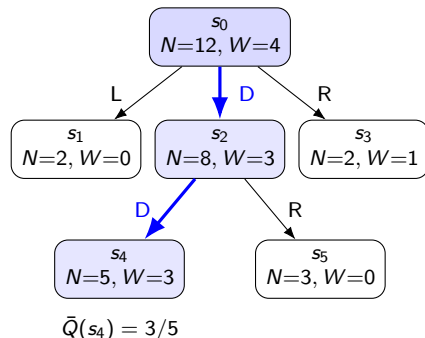
- States are cells in a 4x4 grid
- Actions: left, down, right, up
- Holes terminate with reward 0
- Goal terminates with reward 1
- All intermediate rewards are 0



Note: this MDP is deterministic unless you turn on slippery transitions; MCTS can handle stochastic transitions as well.

Game-tree view

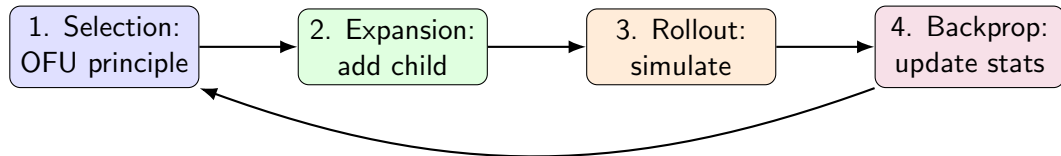
- Root node = current state s_0 where we must act now.
- Each child records the action that created it, `action_from_parent`; the edge reward is folded into the simulation return.
- For each node v , we track:
 - state: FrozenLake cell;
 - parent: pointer for backpropagation;
 - children[action]: expanded successors;
 - untried_actions: actions not yet expanded from this state;
 - $N(v) = \text{visits}$; $W(v) = \text{value_sum}$;
 - $\bar{Q}(v) = W(v)/N(v)$.



In this deterministic tutorial, each action has one child; stochastic domains may sample multiple outcomes over repeated simulations.

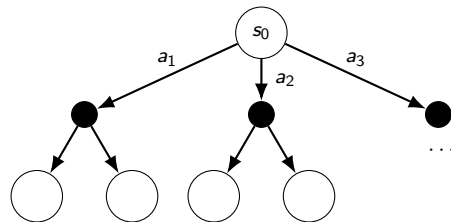
The four MCTS phases

At each current state, MCTS performs multiple simulations, each consisting of four phases:



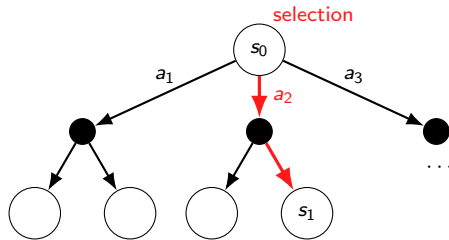
Repeat multiple times, then pick the best root action $\leftrightarrow$ the one visited the most.

One MCTS simulation on a tree



Start from existing statistics.

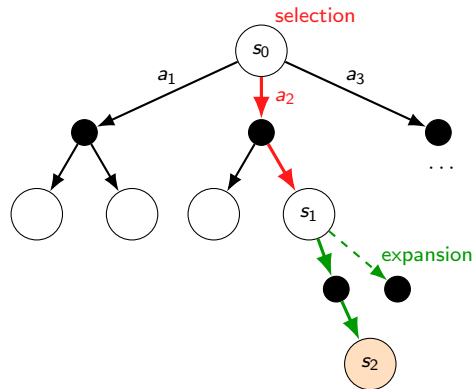
One MCTS simulation on a tree



Follow UCB until an expandable node.

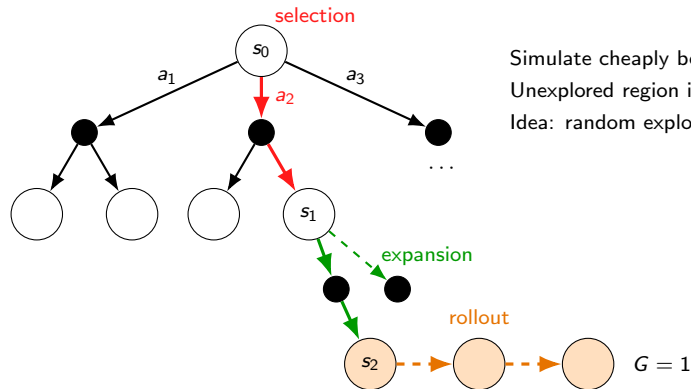
Example: $a_2 = \arg \max_{a \in A(s_0)} UCB(a|s_0)$

One MCTS simulation on a tree



Add exactly one untried action.

One MCTS simulation on a tree

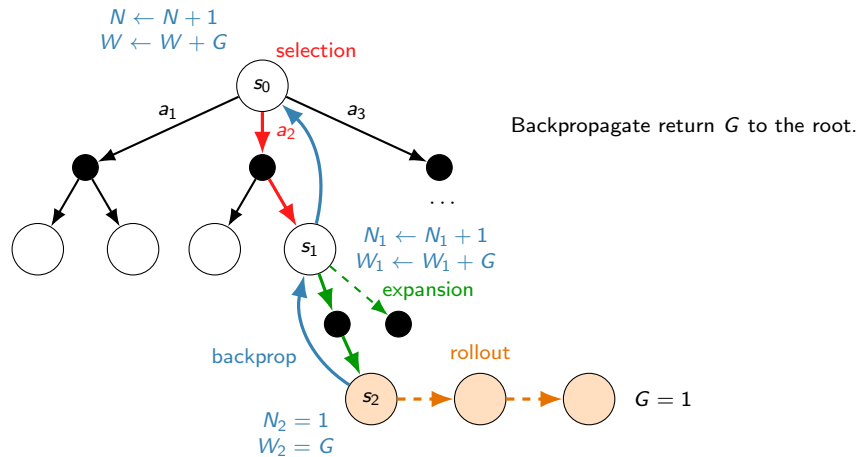


Simulate cheaply beyond the tree.

Unexplored region in search space $\rightarrow$ cannot use UCB!

Idea: random exploration.

One MCTS simulation on a tree



Selection: Optimism in the Face of Uncertainty

Idea: Select action with best mean reward? Not quite, we need to explore less-visited actions!

We use the Upper Confidence Bound (UCB) score to balance exploitation and exploration

↪ **OFU** principle

Child UCB score

$$\text{UCB}_i = \bar{Q}_i + c \sqrt{\frac{\log N_p}{N_i}}$$

- $\bar{Q}_i$: the child i 's mean return W_i/G_i (exploitation term)
- N_i : number of visits to child i
- N_p : number of parent visits
- c : positive constant

Unvisited children receive infinite score so they are tried at least once first

Action	Visits	Mean return
Right	20	0.55
Down	4	0.50
Left	2	0.10
Up	2	0.00

- Right currently looks best
- Down has fewer samples and may still be competitive
- Left and Up may be bad, but one sample is generally not enough evidence
- Exploration bonuses shrink as visits increase

UCB intuition

Action	Visits	Mean return	UCB
Right	20	0.55	0.79
Down	4	0.50	1.03
Left	2	0.10	0.85
Up	2	0.00	0.75

- Right currently looks best
- Down has fewer samples and may still be competitive
- Left and Up may be bad, but one sample is generally not enough evidence
- Exploration bonuses shrink as visits increase

UCB intuition

Action	Visits	Mean return	UCB
Right	20	0.55	0.79
Down	4	0.50	1.03
Left	2	0.10	0.85
Up	2	0.00	0.75

- Right currently looks best
- Down has fewer samples and may still be competitive
- Left and Up may be bad, but one sample is generally not enough evidence
- Exploration bonuses shrink as visits increase

Expansion and rollout

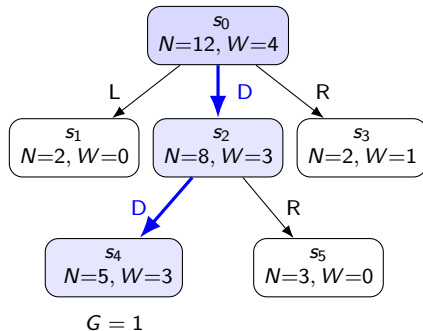
- Expansion adds one untried action from the selected node
- The immediate transition reward belongs to the expanded edge
- Rollout estimates the remaining future return cheaply
- In sparse-reward domains, purely random rollouts often return zero
- The reference solution uses a small amount of heuristic guidance during rollout, while the starter exercise can begin with uniform random rollouts

Backpropagation

- Suppose a simulation returns G
- Walk from the leaf to the root
- For every visited node:

$$N \leftarrow N + 1, \quad W \leftarrow W + G$$

- Future UCB decisions now include this evidence

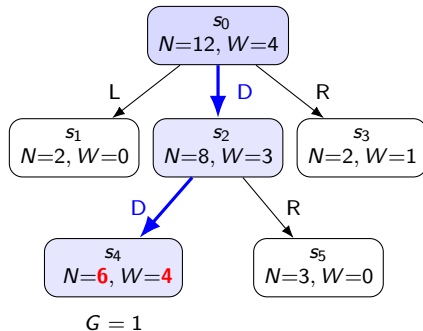


Backpropagation

- Suppose a simulation returns G
- Walk from the leaf to the root
- For every visited node:

$$N \leftarrow N + 1, \quad W \leftarrow W + G$$

- Future UCB decisions now include this evidence

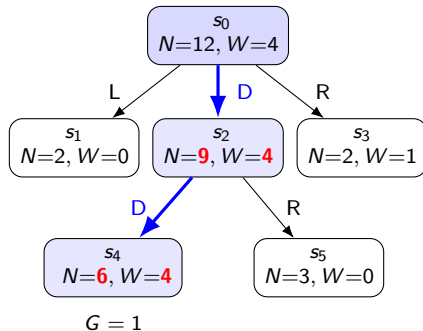


Backpropagation

- Suppose a simulation returns G
- Walk from the leaf to the root
- For every visited node:

$$N \leftarrow N + 1, \quad W \leftarrow W + G$$

- Future UCB decisions now include this evidence

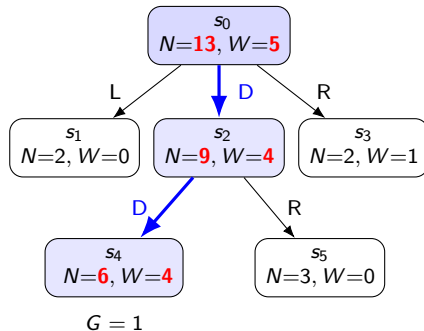


Backpropagation

- Suppose a simulation returns G
- Walk from the leaf to the root
- For every visited node:

$$N \leftarrow N + 1, \quad W \leftarrow W + G$$

- Future UCB decisions now include this evidence



Implementation skeleton

```
for simulation in range(iterations):  
    node = select(root)  
    if node can be expanded:  
        node, reward0 = expand(node)  
    rollout_return = rollout(node.state)  
    total_return = reward0 + gamma * rollout_return  
    backpropagate(node, total_return)  
  
action = best root child action
```

- The starter file leaves only these core pieces as TODOs
- The complete solution adds diagnostics for policy grids and animations

Frozen lake is not necessarily trivial



- Reward is observed only at the goal
- Holes create multiple terminal failures
- With very few simulations, MCTS often lacks evidence for the correct route
- With tens or hundreds of simulations per action, it should become reliable

Hands-on exercise

1. Implement transition sampling (`_sample_next_state`)
2. Implement UCB score (`_ucb_score`)
3. Implement selection and expansion (`_select` and `_expand`)
4. Implement rollout and backpropagation (`_rollout` and `_backpropagate`)
5. Combine the phases in `plan` method
6. Run tests and observe policy behavior in the notebook

Goal: replace the TODO placeholders with actual implementation

Extras:

- Sweep UCB constant c and rollout depth
- Replace random rollouts with a heuristic or learned rollout policy
- Turn on slippery transitions and see what happens

Takeaways

- MCTS turns a simulator into an anytime action-selection algorithm
- UCB balances exploitation of good actions and exploration of uncertain actions
 - Wait to see its full strength during the lecture on Stochastic Bandits!
- Sparse rewards require enough simulations and often better rollout policies
- Policy and trajectory visualisations are useful debugging tools